Introduction to NLP for Data Practitioners

Course Overview

Course Overview

Hi everyone. My name is Mihaela Danci, and welcome to my course, Introduction to Natural Language Processing for Data Practitioners. Are you curious about how digital assistants understand our queries or how language translation apps seamlessly convert text from one language to another? These examples represent just a small portion of what natural language processing can achieve. This course is a quick introduction to developing natural language processing solutions with Python. Some of the major topics that we'll cover include understanding the applications of natural language processing, exploring the components of a natural language processing pipeline, and identifying common challenges in this field. By the end of this course, you have a solid foundation to get started implementing end-to-end natural language processing pipelines using Python. Before beginning the course, you should be familiar with Python and machine learning concepts. From here, you should feel comfortable diving into natural language processing with courses on text data cleaning techniques, preprocessing techniques, and sentiment analysis. I hope you'll join me on this journey to learn how to extract insights from text using natural language processing techniques with the Introduction to Natural Language Processing for Data Practitioners course, at Pluralsight.

Understanding Natural Language Processing (NLP)

Introduction

Welcome everyone. This is the Introduction to Natural Language Processing for Data Practitioners course, and I'm thrilled to be your guide on this journey as we explore how NLP, short for natural language processing, is transforming industries and influencing data-driven decisions. Have you ever wondered how certain applications translate from one language to another, how spelling correction and chatbot work? These are just a few applications of natural language processing. From healthcare and finance to marketing and beyond, natural language processing is the key that unlocks the insights hidden within the vast

amounts of unstructured text data. Together, we are going to explore its applications, dive into its history, and understand why it's becoming an indispensable tool in the hands of data professionals. We'll examine the fundamentals of natural language that are crucial for NLP and begin hands-on learning with Jupyter notebooks focusing on written text.

What Is NLP?

Before we dive into coding, let's understand what NLP is and its significance. Natural language is the natural that we, humans, use to communicate with each other. It's complex and intuitive to us, as speakers and listeners. It includes spoken words, written text, and even non-verbal cues like gestures. Unlike us, computers don't distinctively understand context or emotion, they think in binaries zeros and ones. The translation between natural language, spoken or written, and programming language is done by natural language processing. In other words, NLP is an automated way to understand, interpret, and generate human-like text, or natural language, by applying algorithms. Natural language processing is a branch of AI that combines the field of linguistics, computer science, and machine learning. The field of NLP began in the 1940s, and in the early years, researchers focused on rule-based system when computer scientists began to explore ways to make computers understand and generate human language. In the 1990s, there was a shift towards statistical methods and machine learning. Instead of trying to teach computers every single rule, researchers started showing them examples. In the 2000 deep learning, particular recurrent neural networks, long short-term memory networks, and later, transformer models revolutionized the field. These models excel at capturing contextual information and have led to breakthroughs in transforming language translation and sentiment analysis. Transformers were introduced in 2017 paper, Attention is all you need, by Vaswani et al and offer an alternative to recurring neural networks. They process language using parallel token analysis, improving efficiency and enabling the training of Large Language Models on large datasets, hence the name. Models Like Google's BERT and OpenAI GPT launched in 2018 used extensive pre-training on diverse datasets to understand context and nuances better. Today, natural language processing is rapidly evolving with research aimed at refining models to comprehend things like sarcasm and cultural references. Whether you are in analytics, data science, or any data-related field incorporating NLP into your toolkit will elevate your capabilities. NLP allows us to extract valuable insights from text data uncovering patterns or correlations that might be challenging to identify via manual analysis. Natural language processing also automate time-consuming tasks, such as text summarization, language translations, or information extraction, as well as enhancing user experiences via the virtual assistance, for example. All those applications can be applied in various industries from healthcare, finance, retail, or manufacturing and beyond.

Natural Language Terminology

Natural languages complex and natural language processing algorithms, they construct it to understand and learn from underlying patterns. Let's revisit some fundamental linguistic concepts. Morphology, the study of word formation, is crucial for accurate language processing. It begins with phonenemes, the smallest sounds you need to differentiate words like the distinct sounds between cat and mat. Phonenemes are essential for tasks, such as speech recognition and speech-to-text transcriptions. By combining multiple phonemes, we build a morpheme, which is the smallest unit of meaning in our words. Take the word, unbelievable. It's made up of three morphemes, un meaning not, believ as the root, and able indicating ability. Morphemes are used to interpret the meaning of words, their variation, and how they relate to each other. Lexemes are variations of morphemes with a shared meaning such as speak, spoke, and spoken. They are key to semantic analysis and text normalization. Natural language processing uses algorithms for morphological analysis like stemming and lemmatization. Stemming simplifies words to their base forms and lemmatization more accurately returns words to their dictionary forms. Syntax is another important element of natural language and is a set of rules organizing words into sentences. It's what makes I have a cat and a dog comprehensible. The structure is simple for us, but complex for computers, which must parse and analyze each word's role within a sentence, tasks achieved through part of speech tagging and statistical methods. Syntax powers everything from voice assistants to grammar checkers, enabling machines to process our request, correct our writing, and even translate languages with precision. Where syntax is structured, semantics allows us to interpret the meaning of words and sentences within context. Advanced models like recurring neural networks and transformers help computers grasp nuances behind phrases, such as I'm feeling blue, discerning emotion from literal interpretation. Sentences formed from words and punctuation build up into paragraphs, documents, and larger corpora which are then deconstructed by NLP algorithms. However, in spoken language, intonation and rhythm add layers of meaning upsetting text, presenting challenges for NLP, such as detecting sarcasm. Later in the course, we'll deep dive into the challenges faced by the NLP systems.

NLP Toolkit

Powerful toolkits, libraries, and frameworks make natural language processing not just visible, but efficient. NLTK is the natural language toolkit is a favorite among Python enthusiasts. It offers over 50 corpora and lexical resources and a range of text processing libraries. For more advanced work in Python, we turn to spaCy. It's crafted for production environments, empowering the creation of applications that process and interpret large volume of text. Java users can leverage CoreNLP

created by Stanford NLP group which contains a suite of tools that provides models and algorithms for a wide range of languages. TensorFlow provides KerasNLP and TensorFlow Text to libraries finetuned for text and NLP tasks. Let's not forget frameworks like Hugging Face transformers which bring state-of-the-art models, like BERT, right to your fingertips, enabling the use of extensively-trained models through minimal coding.

Demo: Text Operations

In the rest of the course, we'll work with NLTK and spaCy. So without further ado, let's install these two libraries and then do several text processing operations, such as dividing sentences into words or identifying which part of speech each word is. If you'd like to follow along, make sure you install Python and Jupyter Notebook on your machine. Switching to Jupyter notebook, let's install the NLTK and SpaCy packages using the pip install command. I already installed them, so instead of running this piece of code, I'm going to comment it and query their versions. These are the latest versions at the time of recording. Next, we import NLTK into the notebook. NLTK has many modules and sub modules, and to process text, we use functions from various modules. To delimit each task, we'll import them as needed. Alright, now let's create a variable called text and sort this piece of text. Ben relocated to Paris last year to pursue his passion. He is currently enrolled in a comprehensive course on natural language processing. The NLTK data package includes a pre-trained punkt tokenizer for English, and we download it. To facilitate our analysis, we break this piece of text into words. For this, we need a function word tokenize from nlt.tokenize module. Next, we apply this function on our text and save the result into a variable, tokens. As soon as we run this piece of code, we see that each word is now an item in a list. Next, let's display the root or the stem for each word. Again, we need to import a new function. PorterStemmer from an nltk.stem. The PorterStemmer is an algorithm used for stemming in English, which means it reduces words to their root form. Next, we create a variable stemmer and assign an instance of the PorterStemmer to perform the stemming process on the words. Then with a for loop, we iterate over each token in the tokens list and apply the stemming algorithm, reducing each word to its stem. The result is taken to the stemmed_words list. Check it out. For example, the word enrolled has been reduced to enroll, getting us the core idea of the word. Now, we move on to part of speech tagging and we determine which word is a noun and which is the verb and so on. First from NLTK, we import pos_tag and then we apply it to our tokens, saving the result into a variable. By printing its content, we see that each word from our text has been tagged with each part of speech. Ben is a singular proper noun, while relocated is a verb in past tense. This function shows us the label for each tag. Let's see how Python recognizes Ben and Paris as being a person and a city respectively. For this, we download maxent_ne_chunker and words. Then we simply apply to our last list and save the result into a new variable entities

following by printing its content. Now, the entities in our text are recognized. Lastly, we explore word similarity using pretrained word embeddings. Word embeddings are a type of word representation where words are mapped to vectors allowing words with similar meaning to have a similar representation. Here, we import the library loading each model and process the words king, queen, and apple. We then calculate a similarity between the embeddings of the words king and queen and between the word embeddings of king and apple. As expected, king and queen show higher similarity due to their contextual closeness.

Summary

We started this module by understanding that natural language processing is an automated way to understand, interpret, and generate human-like text by applying algorithms. Next, we learn about natural language processing evolution from rule-based systems to advanced deep learning. We then explore linguistic fundamentals such as morphology, syntax, and semantics, key concepts that enable computers to process human language. Finally, we jump to Jupyter Notebook where we applied tokenization stemming and part of speech tagging on a small paragraph. In the next module, we will focus on the entire NLP pipeline from data collection to model validation. I'm looking forward to seeing you there.

Exploring the NLP Pipeline

Introduction

Welcome back. The natural language processing pipeline consists of a series of steps that enable computers to understand, interpret, and generate human language. The pipeline begins with text acquisition where text data is gathered from various sources such as books, online content, or speech transcripts. Next come data cleaning which involves steps like removing unnecessary elements like HTML tags or correcting typos. Following that, we have text preprocessing. In this stage, we prepare the text for analysis, and this phase includes tokenization, breaking the text into words, and normalization which converts the text into a more consistent form like lower casting and stemming. The process then moves to feature extraction or feature engineering where we translate textual information into numeric format that computers can interpret using techniques like bag of words. Afterward, we proceed to modeling. At this stage, we apply algorithms to analyze the text. Applications might range from classification tasks like sentiment analysis or clustering tasks like token modeling. After modeling, we evaluate the results using metrics like accuracy, precision, and recall. Based on the evaluation, we iterate on our models

and preprocessing steps to improve performance. The final step involves deploying our natural language processing model into a real-world application and maintaining it. This includes monitoring its performance and updating it as needed to adapt to new data or requirements. In the remaining sections of these modules, we'll deep dive into each element. At the same time, we will gain hands-on experience while tackling a classification problem. Our objective is to effectively categorize news articles based on their content.

Data Acquisition

In any NLP project, data is as crucial as building materials are to a house. Data acquisition involves sourcing relevant text from various places like digital libraries, websites, and media transcripts. The aim is not just volume, but also the variety, seeking diverse languages, styles, and context to enrich our NLP systems. The ideal situation will be to have the needed data pre-collected and formatted via our own organization. However, this is often not the case. To gather the necessary data, we might explore public datasets, utilize web scraping, employ APIs, or even conduct manual collections. Data augmentation also plays a role enhancing our dataset by making slight variation to the existing text, such as using synonyms or rephrasing to increase its volume while keeping the meaning intact. For inaccessible or sensitive data like financial or medical records, we turn to synthetic data creation. This involves algorithms generating entirely new text often via GANs, where two AI models collaborate, one generating new data and the other assessing its authenticity. Synthetic data offers benefits such as privacy protection, increased diversity to minimize bias, and large volumes of data suitable for deep learning without extensive data gathering. Once we compile our data, we move on to the next stage, data cleaning.

Demo: Data Acquisition

It's time to import our data into Jupyter Notebook for our classification project. We'll be working with the 20 newsgroup dataset, a collection of newsgroup documents across 20 different topics from space exploration to baseball. Our objective is to navigate the NLP pipeline with this dataset, ultimately, building a model to classify posts into categories. Moving to Jupyter Notebook, we start by importing the fetch_20newsgroups function from scikit-learn datasets. This function pulls around 20,000 documents. At this point, we also import the pandas library needed for data manipulation. Next, we load the data into a variable named post. We'll work with all the available subsets while analyzing the articles from electronics and space categories. Let's examine the data's structure by printing its keys. It includes data, the post text, target_names, the actual category names, target, numerical indices for categories, and a comprehensive dataset description. Next, we take a peek at a single

post. Using the print function, we access the post content from the data attribute. Note that the document contains headers and codes which we'll address in the cleaning phase. Finally, we create a dataframe df containing just the data in the target category. Displaying the first five entries is done with the print and head functions, while the shape function indicates that we have nearly 2,000 posts in total.

Data Cleaning and Preprocessing

In the cleaning phase of the NLP pipeline, we ensure the data is filtered and validated to guarantee accuracy, relevance in the absence of errors or duplicates. Raw data typically carries noise and needed formatting mistakes or irrelevant details. Our initial step is to clean and standardize the data, preparing it for the next stages. Cleaning involves addressing issues like spelling, grammar, typographical errors, or whitespaces. Consistency is also key involving adjustments such as standardizing capitalization and date formats. We also eliminate noise by discarding irrelevant elements like metadata or non-textual content that might disrupt our analysis. Natural language processing algorithms deconstruct natural language to understand and learn from underlying patterns. We already had a glimpse over the pre-processing steps in the previous module. The first preprocessing step is breaking down the text into sentences and then into words or tokens, a process called tokenization. After tokenization, we remove stop words or common words like and or the. Stop words are removed because they are typically irrelevant in understanding the essence of the text. Next, we apply stemming and lemmatization algorithms to reduce words to their base forms. For instance, transforming better to good. Lemmatization is more accurate and needs more resources while stemming is faster, but less precise. The final steps include part of speech tagging and named entity recognition. Part of speech tagging categorizes each token according to its grammatical role, such as noun or verb, while named entity recognition identifies and classifies key information in the text, like names of people, organizations, or locations which aid in extracting meaning and context from the data.

Demo: Data Cleaning and Preprocessing

Let's proceed to clean and preprocess our documents, an essential step before applying the classifier in Jupyter Notebook. We'll remove headers, whitespace, punctuation, and then focus on tokenization and lemmatization. Continuing from where we left off, I updated our list of libraries to include those essentials for data cleaning and preprocessing. We've also noticed earlier that some articles include headers while others may include other metadata. To streamline our data, we use the remove parameter to filter this out. Next, we define a function named

clean_text which will streamline the cleaning and preprocessing steps. This function takes text as input, process it through our define operations, and returns the clean_text. Our process begins with tokenization, splitting the text into individual words using the word_tokenize function. The result is stored in tokens. We then remove tokens that are not purely letters like numbers and punctuation by applying the isalpha function. In this way, we focus our analysis on actual words. The third step involves converting all tokens to lowercase to treat words uniformly. Then, we address stop words, common words like and, the, which are typically irrelevant in understanding the essence of the text. We import the predefined english stop words list from NLTK, and we eliminate this from our tokens. The next process is Lemmatization. By applying the WordNetLemmatizer algorithm, the words are reduced to their base form. This helps in processing different forms of a word as a single entity. The final step is rejoining the clean tokens into a string, ensuring that there is only one space between each word. Now let's apply the clean_text function to our data frame text column, storing the output in a new clean_text column. By printing the top rows, we can see the transformation. For analysis, we'll sort the clean_text and labels in a new data frame. And that's it. We've transformed raw text into a list of standardized word forms ready for our classification model.

Feature Extraction

Feature extraction or feature engineering converts pre-processed text into a numerical format for NLP algorithms. These algorithms prefer numeric data so we transform text features into numeric vectors. Here's how. One method is one-hot encoding. Each word is encoded as a unique vector represented as the one in its position against the entire vocabulary, which is represented as zeros. However, thi s creates a sparse matrix large and mostly zeros for big vocabularies, which is inefficient. The bag-of-words model counters sparsity by counting word frequencies without considering grammar or word order useful for classifying texts that share similar words. The theory is that the more frequent a word is, the more important it might be to the text. Despite its usefulness, this model loses context because it ignores word order, making these two sentences appear the same in the model. The bag-of-n-grams model extends this by considering a sequence of n words together capturing more context. Increasing n provides richer detail, but it can lead to a feature explosion risking model overfitting. TF-IDF, short for term frequency inverse document frequency, helps identifying the most significant words in a document. TF counts how often a word appears in a given document, while IDF measures the importance of the term across a corpus. Mathematically, TF-IDF is their product, assigning a score that reflects a word's importance in the document relative to a collection of documents. A higher TF-IDF score suggests a greater relevance. On the other hand, IDF suggests rarity implying that common words are less informative. Words like is, are are not important in a document, even though they

occur frequently. Word embeddings are another type of representation in natural language processing where words from the vocabulary are mapped to vectors of real numbers in a predefined space. Each word is represented by a dense vector that capture its semantic meaning based on the context in which it appears. In this way, similar words have a similar representation. These embeddings are learned from large corpora of text data. One popular model for creating word embeddings is word-to-vec.

Demo: Feature Extraction

In the next few minutes, we'll vectorize our data using bag-of-words and TF-IDF techniques. Let's dive into Jupyter Notebook for this process. Before feature extraction, it is essential to split our dataset into training and testing sets. We define X as our clean_text column which contains the preprocessed data and serves as input for our model. Y will represent the target variable for prediction. We then import train_test_split from scikit-learn model selection to create our training and testing sets using a 75/25 split. To maintain consistency across runs, we send a random state to 1. We check the dataset shape to understand the number of elements in each set. 1,478 observations will be used to train the model while 493 will be used to test its accuracy. Now, we are ready to transform the text data into vectors. We begin with the Bag-of-words model by importing CountVectorizer from scikit-learn. CountVectorizer counts how many times each word appears in the vocabulary converting tokens into a bag-of-words metrics. Next, we instantiate it and set a min_df parameter to 10 to include terms appearing at least in 10 documents. We then fit and transform the training data. The Fit and transform method fits the model or learns the vocabulary of the training set, and then it transforms the training data into a matrix of token counts. At the same time, we apply the same CountVectorizer to the test data. It's important to note that we only transform and not fit the test data. This means that the test data will be transformed into a matrix of token counts using the vocabulary learn from the training data preventing data leakage. Let's display a few feature names and counts from the training data. For readability, we convert our token count metrics to a pandas data frame setting the columns to feature names from CountVectorizer. This matrix has row representing our documents and columns for each unique word in our vocabulary. Next, we explore ngrams by adjusting the ngram_range parameter to 2 and 2 in CountVectorizer to generate by grams or two-word combinations. This parameter sets the lower and upper boundary of the range of n values for different word n grams. Now when we run this, we see the background with terms like air force. Moving to TF-IDF representation, we use TF-IDF vectorizer from scikit-learn. This tool computes the TF-IDF score for each word. We create a TfidfVectorizer instance, restricting the minimum and maximum document frequency. Next, we apply fit_transform to the training data and transform to the test data. Finally, like the bag-of-words, we display the TF-IDF matrix using a pandas data frame. Each column in this matrix

corresponds to a vocabulary term with the values representing each term's TF-IDF score in the document. This process effectively transforms our clean text into vectors, preparing it for use in a classifier.

Data Modeling and Evaluation

In our journey through natural language processing, we came across various models and algorithms. These range from rule-based systems and machine learning to advanced neural networks, each tailored to solve specific NLP challenges. Text classification is a fundamental NLP task where text is categorized into predefined groups. For instance, classifying emails as spam or non-spam or organizing articles by topics can be effectively handled by algorithms, such as Naive Bayes, support vector machines, and neural networks. Sentiment analysis, another essential technique, discerns emotion or sentiments within text. This is especially important for brand monitoring and analyzing customer feedback. Techniques like logistic regression, long short-term memory networks, and other deep learning models are commonly used here. Name entity recognition is about identifying and categorizing key elements in text, such as names of people, places, or organizations. Effective algorithms for name entity recognition include conditional random fields and transformer-based models like BERT. Speech recognition, the conversion of spoken language into text, has evolved from using hidden Markov models to now employing deep neural networks, particularly long short-term memory networks. This generation has seen significant advancements with LSTM networks and transformer models like OpenAI's GPT series which produces remarkably human-like text. Machine translation has transitioned from algorithms like statistical machine translation to more advanced neural machine translation models, such as seq2seq and transformers, setting new standards in the field. After we complete the modeling phase in our NLP project, the next step is to evaluate our model's performance. We use various metrics for this purpose including accuracy, precision, recall, and F-score. Accuracy measures the proportion of correct prediction among the total number of cases evaluated. Precision looks at the proportion of two positive predictions in the total positive predictions made by the model via recall or sensitivity, assesses the proportion of actual positive correctly identified. This evaluation phase is not just a one-time process, but part of an iterative circle. Based on the results we obtain, we define our models and preprocessing techniques to enhance their performance. This might involve adjusting parameters, choosing different algorithms, or revising our data cleaning methods. The final phase of our NLP project is the deployment of the model into a real-world application. The form of deployment varies based on specific use cases ranging from integration into software application to utilization in data analysis or embedding within websites. Post deployment. The model requires regular maintenance to adapt to changing language use, user behavior, and context ensuring the model remains effective and relevant in real-world

applications.

Demo: Data Modeling and Evaluation

We've successfully converted our data into feature vectors, setting the stage to train and evaluate the Naive Bayes classifier. This classifier is particularly suited for count vectorized data as it expects integer inputs. In Jupyter Notebook, we begin by importing the multinomial naive_bayes model from Naive Bayes module. Along with this, we also import the metrics module to assess the model's performance. Next, we instantiate the multinomial Naive Bayes classifier, assigning it to the variable NB. We then train a classifier using the fit method with X_train_counts, our count vectorizer output as features and y_train as labels. This process enables the model to learn the correlation between features and labels. After training, we use the predict method of our NB classifier to forecast labels for the vectorized test data, X_test_counts. The predicted labels are stored in the y_pred variable. To evaluate the model, we employ the accuracy_score function from the metrics module. This function compares the actual labels, y_test, against our predicted labels, y_pred, calculating the model's accuracy. Our model demonstrates high accuracy in this instance. Additionally, we compute the Confusion Matrix. This reveals that 7 electronics documents were misclassified as space, and 42 space documents were inaccurately classified as electronics, providing us with insights of the model's classification errors.

Challenges in NLP

While NLP has great potential, it also faces several challenges. One of the main challenges in NLP is managing words with multiple or related meanings. For example, bat might mean an animal or sports equipment and light can imply either low weight or brightness. This kind of ambiguity can complicate text interpretation for NLP systems. To address this, NLP uses various disambiguation techniques. One method involves algorithms that use context to determine a word's meaning. Another uses machine learning models trained on large datasets to recognize patterns and contextual meanings. Sentiment analysis is another area where context plays a vital role. For example, in the sentence, The movie was incredibly light, the word light might imply a positive sentiment if it means the movie was enjoyable and not serious. Detecting sarcasm and irony in text is also challenging. These forms of expression often rely on tone which can be hard to convey through text. As a result, an NLP system might misinterpret a sarcastic comment as a serious statement. NLP must also handle text in multiple languages and dialects. Each language has its own set of rules and nuances, and dialects can vary significantly even within the same language. This diversity presents a substantial challenge for developing universally effective NLP

systems. Recognizing and respecting cultural and linguistic diversity is essential in NLP. Systems must be trained on diverse datasets to ensure that they are not bias towards one language or cultural perspective. Lastly, we must consider ethical issues like content moderation, bias, and fairness. NLP systems can propagate bias if they are trained on biased data. Ensuring these systems are fair and neutral is a significant ongoing challenge in the field. And with this, we have arrived at the end of this course. In this module, we explored the end-to-end NLP pipeline while tackling a classification problem. We begin by exploring various methods for acquiring data essential for NLP projects. Then, we discovered data cleaning and preprocessing techniques. Next, we transform text into a numerical format that classifiers can understand using the bag of words and TF-IDF models. Last, but not least, we applied the Naive Bayes classifier to our data and evaluated this performance. Finally, I'd like to say thank you for watching this course, and if you have a moment, I'd appreciate if you'd leave a rating. I hope you enjoyed this course as much as I enjoyed making it. I'd love to hear from you, and don't hesitate to reach out to me on LinkedIn at the address on the screen.